

EXP 6: Implement a program to retrieve similar products or movies based on high-dimensional vector embeddings

Program:

```
# EXP 6: Similar Product Retrieval using FAISS

!pip install -q kagglehub[pandas-datasets] faiss-cpu transformers torch pandas

import kagglehub
from kagglehub import KaggleDatasetAdapter
import pandas as pd
import torch
import numpy as np
import faiss
from transformers import AutoTokenizer, AutoModel
file_path = "amazon_products.csv"

df = kagglehub.load_dataset(
    KaggleDatasetAdapter.PANDAS,
    "asaniczka/amazon-products-dataset-2023-1-4m-products",
    file_path
)

print("Dataset Loaded Successfully")
print(df.head())
df = df.head(1000)
TEXT_COLUMN = df.columns[0]
item_texts = df[TEXT_COLUMN].astype(str).tolist()

print("\nTotal items:", len(item_texts))
print("Sample item:", item_texts[0])

device = "cuda" if torch.cuda.is_available() else "cpu"
model_name = "sentence-transformers/all-MiniLM-L6-v2"
tokenizer = AutoTokenizer.from_pretrained(model_name)
encoder = AutoModel.from_pretrained(model_name).to(device)
encoder.eval()

def encode_texts(texts, batch_size=16):
    all_embeddings = []

    with torch.no_grad():
        for i in range(0, len(texts), batch_size):

            batch = texts[i:i+batch_size]

            tokens = tokenizer(
                batch,
                padding=True,
                truncation=True,
                return_tensors="pt"
            ).to(device)
```

```

        outputs = encoder(**tokens)
        embeddings = outputs.last_hidden_state.mean(dim=1)
        embeddings = embeddings.cpu().numpy().astype("float32")
        all_embeddings.append(embeddings)
        return np.vstack(all_embeddings)

item_embeddings = encode_texts(item_texts)
faiss.normalize_L2(item_embeddings)
print("\nEmbeddings Shape:", item_embeddings.shape)
embedding_dim = item_embeddings.shape[1]
index = faiss.IndexFlatIP(embedding_dim)
index.add(item_embeddings)
print("FAISS index built with", index.ntotal, "items")
def get_similar_items(item_id, k=5):
    query_vector = item_embeddings[item_id:item_id+1]
    scores, indices = index.search(query_vector, k)
    return scores[0], indices[0]
query_id = 0
scores, idxs = get_similar_items(query_id, k=5)

print("\nQUERY ITEM:")
print(item_texts[query_id])
print("\nSIMILAR ITEMS:")

for score, idx in zip(scores, idxs):
    print(f"-> {item_texts[idx]} (score={score:.3f})")

```

Output:

```
23-07-23 0:00:16.7 /tmp/ipykernel_3477/3359871476.py:19: DeprecationWarning: Use dataset_load() instead of load_dataset(). load_dataset() will be removed in a future version.
df = kagglehub.load_dataset(
Warning: Looks like you're using an outdated 'kagglehub' version (installed: 0.3.13), please consider upgrading to the latest version (1.0.0).
Downloading from https://www.kaggle.com/api/v1/datasets/download/asaniczka/amazon-products-dataset-2023-1-4m-products?dataset_version=17&file_name=amazon_products.csv...
100%|██████████| 99.2M/99.2M [00:01<00:00, 81.5MB/s]Extracting zip of amazon_products.csv...

Dataset Loaded Successfully
   asin                                     title \
0  B014TMV5YE  Sion Softside Expandable Roller Luggage, Black...
1  B07GDLCOXV  Luggage Sets Expandable PC+ABS Durable Suitcas...
2  B07XSCCZYG  Platinum Elite Softside Expandable Checked Lug...
3  B08MVF6GJM  Freeform Hardside Expandable with Double Spinn...
4  B01D1LKZBA  Winfield 2 Hardside Expandable Luggage with Sp...

   imgUrl \
0  https://m.media-amazon.com/images/I/815d0KQTV...
1  https://m.media-amazon.com/images/I/81b01b7vfe...
2  https://m.media-amazon.com/images/I/71EA35zVJB...
3  https://m.media-amazon.com/images/I/91kRN1OyI...
4  https://m.media-amazon.com/images/I/61NJoazCP9...

   productURL  stars  reviews  price  listPrice \
0  https://www.amazon.com/dp/B014TMV5YE  4.5      0  139.99      0.00
1  https://www.amazon.com/dp/B07GDLCOXV  4.5      0  169.99     209.99
2  https://www.amazon.com/dp/B07XSCCZYG  4.6      0  365.49     429.99
3  https://www.amazon.com/dp/B08MVF6GJM  4.6      0  291.59     354.37
4  https://www.amazon.com/dp/B01D1LKZBA  4.5      0  174.99     309.99

   category_id  isBestSeller  boughtInLastMonth
0           104           False                2000
1           104           False                1000
2           104           False                 300
3           104           False                 400
4           104           False                 400

Total items: 1000

sample item: B014TMV5YE
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret 'HF_TOKEN' does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
config.json: 100%|██████████| 612/612 [00:00<00:00, 109kB/s]
tokenizer_config.json: 100%|██████████| 350/350 [00:00<00:00, 64.5kB/s]
vocab.txt: 232k? [00:00<00:00, 14.9MB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
tokenizer.json: 466k? [00:00<00:00, 42.7MB/s]
special_tokens_map.json: 100%|██████████| 112/112 [00:00<00:00, 17.3kB/s]
model.safetensors: 100%|██████████| 90.9M/90.9M [00:01<00:00, 270MB/s]
Loading weights: 100%|██████████| 103/103 [00:00<00:00, 1575.80B/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key | Status | |
-----+-----+---+
embeddings.position_ids | UNEXPECTED | |

Notes:
- UNEXPECTED : can be ignored when loading from different task/architecture; not ok if you expect identical arch.

Embeddings Shape: (1000, 384)
FAISS index built with 1000 items

QUERY ITEM:
B014TMV5YE

SIMILAR ITEMS:
-> B014TMV5YE (score=1.000)
-> B01MA9PLMG (score=0.882)
-> B01H02DYSM (score=0.864)
-> B084V4SNMX (score=0.861)
-> B01ETSEHL4 (score=0.854)
```